PROYECTO INTEGRAL / AVANCE 1 / 1.00 PUNTO

CampusHub Labs Semana 13

Guía de ejecución, revisión y sustentación

Luis Brandon Laurente Macha · U18311719

El primer avance conecta tres piezas: un núcleo de reglas en JavaScript, una API simulada que guarda datos y un panel Angular para administrar laboratorios. Esta guía acompaña al código y a la colección de Postman incluidos en el ZIP.

Pieza	Qué resuelve
core-js	Validaciones, clases, cruces de horarios y estadísticas reutilizables.
api-mock	json-server 0.17.4, datos persistentes en db.json y colección HTTP.
angular-admin	Catálogo con CRUD, formulario reactivo, detalle y consulta de reservas con Bootstrap.

Qué corresponde entregar ahora

Código del avance 1 en el repositorio del grupo, colección exportada, README actualizado y etiqueta avance-1. El enlace se registra en el aula virtual. React se incorpora a partir de la semana 14.

Datos preparados para la demostración

Usuarios	Laboratorios	Reservas
2 administradores 6 estudiantes	8 espacios 4 sedes	20 solicitudes 7 pendientes, 7 aprobadas, 6 rechazadas

Los registros de la base son ficticios. El panel de esta semana abre directamente; todavía no se implementa una pantalla de login.

Antes de presentar

Completar los integrantes reales del grupo, confirmar la aprobación del dominio con el docente y publicar el repositorio y la etiqueta. La generación del ZIP no realiza esas acciones externas.

Fuente del alcance: PDF del profesor, secciones 7 y 13; requisitos en páginas 5-7 y entregables en páginas 12-13. No se exige un Word o una presentación como archivo de este avance.

Ejecutarlo en VS Code

1. Abrir la carpeta correcta

Extrae el ZIP completo. En VS Code elige **Archivo > Abrir carpeta** y selecciona **CampusHub_S13**. Debes ver README.md, package.json, core-js, api-mock y angular-admin en la misma raíz.

2. Comprobar e instalar

En **Terminal > Nueva terminal**, ejecuta cada línea. Usa Node 22.12 o posterior de la rama 22, o Node 24.x. La instalación inicial necesita Internet; no requiere Angular global.

```
node --version
npm --version
npm ci
npm start
```

3. Abrir el panel

Espera el mensaje de compilación completada y entra a **http://localhost:4200**. La API responde en **http://localhost:3000**. Mantén la terminal abierta. Con **Ctrl+C** detienes ambos procesos.

4. Verificar el proyecto

```
npm test
npm run test:api
npm run build
```

npm test ejecuta las 30 pruebas del núcleo. test:api comprueba 15 comportamientos HTTP usando una copia temporal. build compila el panel para detectar problemas de TypeScript y plantillas.

Si aparece un problema

Mensaje o situación	Qué hacer
npm.ps1 bloqueado en PowerShell	Cambiar la terminal a Command Prompt o usar npm.cmd en los comandos.
No encuentra package.json	Abrir la raíz CampusHub_S13, no solo la carpeta src.
La API no se puede conectar	Mantener npm start activo; alternativamente usar npm run api y npm run admin en dos terminales.
Puerto 3000 o 4200 ocupado	Detener con Ctrl+C la ejecución anterior de este proyecto y volver a iniciarlo.
Quiero volver a los datos iniciales	Detener la API. Ejecutar npm run restaurar-datos -- --confirmar. Reemplaza los cambios del JSON.

Cada requisito tiene un archivo

La rúbrica suma 1.00 punto, distribuido en cuatro criterios de 0.25. Esta tabla sirve para comprobar la cobertura y para localizar evidencia durante la exposición.

Criterio	Evidencia en el proyecto
Núcleo JS	core-js/validadores.js, modelos.js, estadisticas.js y core.test.js.
API simulada	api-mock/db.json, package.json y CampusHub_Avance1.postman_collection.json.
CRUD en Angular	components/laboratorio-lista, laboratorio-detalle y laboratorio-formulario; services/campus.service.ts.
Elementos Angular	app.routes.ts, modelos.ts, carpeta shared, plantillas con @for/@if/ngClass y Bootstrap en angular.json.

Rutas del panel

Dirección	Vista
/laboratorios	Catálogo, búsqueda, filtro y eliminación.
/laboratorios/nuevo	Creación con validaciones.
/laboratorios/:id/editar	Edición de un laboratorio existente.
/laboratorios/:id	Detalle, equipamiento y reservas asociadas.
/reservas	Consulta y filtros de las solicitudes.
Cualquier otra ruta	Página no encontrada (404).

¿Dónde están TypeScript y Bootstrap?

TypeScript: archivos .ts del panel. Las interfaces de Usuario, Laboratorio y Reserva están en src/app/modelos.ts. Definen los campos que espera el código; no sustituyen la validación en ejecución.

Bootstrap: dependencia 5.3.8 en angular-admin/package.json. Su CSS se incorpora en angular-admin/angular.json, dentro de build.options.styles. Busca bootstrap.min.css. En las plantillas puedes señalar row, col-md-*, btn, table, form-control y alert.

La interfaz también demuestra el núcleo

Las sedes únicas vienen de Set; los contadores de reservas usan reduce; el detalle suma horas aprobadas con Map y el listado muestra un ranking. La copia a src/core se realiza antes de arrancar o compilar.

JavaScript explicado desde cero

Piensa en core-js como una caja de reglas que distintas pantallas pueden compartir. No depende de Angular; por eso se puede comprobar directamente con Node.

Concepto	Idea sencilla y uso real
Regex	Un molde para textos. El correo debe terminar en @utp.edu.pe; el código debe tener U y 8 dígitos.
Clase y constructor	Un molde para objetos. new Reserva(datos) crea una reserva y comprueba que sus datos básicos sean válidos.
Getter y método	reserva.horario calcula un texto al leerlo. reserva.duracionHoras() ejecuta una acción y devuelve una duración.
reduce	Reúne una lista en un resultado. Cuenta reservas en un objeto con tres contadores.
Map y Set	Map relaciona laboratorio con horas aprobadas. Set elimina sedes repetidas.
sort y spread	El ranking ordena una copia. Así no cambia el arreglo original del catálogo.
Closure y padStart	La función generadora recuerda un contador; padStart añade ceros para formar RES-0001.

La regla de horarios

Dos reservas activas chocan si son del mismo laboratorio, del mismo día y sus intervalos se superponen. Las rechazadas no bloquean. Para evitar dobles reservas, cada inicio debe ser anterior al final del otro intervalo.

Primera reserva	Segunda reserva	Resultado
08:00 a 10:00	09:00 a 11:00	Se cruzan.
08:00 a 10:00	10:00 a 11:00	Son contiguas; se permite.
Lab 1, 08:00 a 10:00	Lab 2, 08:00 a 10:00	Distinto laboratorio; se permite.

Qué protegen las pruebas

Comprueban correos y códigos inválidos, fechas imposibles, años bisiestos, límites de capacidad, cruces, reservas contiguas, estadísticas y el generador. También revisan que la semilla tenga cantidades correctas y referencias válidas.

Ocupación significa suma de horas aprobadas del conjunto consultado. No se presenta como porcentaje de uso ni disponibilidad en tiempo real. El contador del closure vive en memoria y no garantiza unicidad entre varios clientes.

Angular y la API trabajando juntos

Al guardar un laboratorio, el formulario produce un objeto. El servicio envía ese objeto a la API. Cuando llega la respuesta, Angular vuelve al catálogo y consulta los datos actualizados sin recargar toda la página.

Acción	Método HTTP	Efecto
Consultar	GET	Obtiene una lista o un detalle.
Crear	POST	Agrega un registro y devuelve un id.
Editar completo	PUT	Reemplaza todos los campos enviados del registro.
Editar parcialmente	PATCH	Cambia solo algunos campos; diagnóstico en Postman.
Eliminar	DELETE	Quita el registro indicado.

Qué hace cada pieza de Angular

HttpClient: envía las solicitudes. **Servicio:** concentra las direcciones y operaciones. **Observable:** representa la respuesta que llegará después; subscribe recibe el éxito o el error.

Formulario reactivo: define campos y reglas en TypeScript. required exige un dato; min y max limitan capacidad a 1-60; el validador propio reutiliza una regex del núcleo para el nombre.

@for: repite filas con un identificador estable. **@if:** muestra carga, error, datos o vacío. **ngClass:** asigna el color de cada estado.

Pipe estadoReserva: transforma el estado en texto con un símbolo. **Directiva appResaltarMantenimiento:** añade un borde a los laboratorios en mantenimiento.

Recorrido para demostrar el CRUD

1. Crear **Lab Demo S13**, sede Lima Centro, capacidad 20 y equipos PC i7, Proyector.
2. Abrir el detalle y comprobar los datos.
3. Editar capacidad a 25 y cambiar a mantenimiento.
4. Comprobar el estado, el borde y la actualización de la tabla.
5. Eliminar ese laboratorio nuevo y comprobar que desaparece.

Por qué no elimina un laboratorio con reservas

El panel conserva las referencias del historial. Si un laboratorio ya tiene reservas, sugiere cambiarlo a mantenimiento. Esta comprobación es del cliente; el backend real resolverá la integridad del lado del servidor.

Postman y verificación

Usar la colección incluida

Abre Postman, importa **api-mock/CampusHub_Avance1.postman_collection.json** y ejecuta la colección en orden con la API activa. baseUrl ya apunta a http://localhost:3000.

Solicitudes	Qué demuestran
01 a 04	GET de catálogo, filtro de sede, búsqueda q y detalle.
05 a 07	POST, PUT y PATCH sobre un laboratorio temporal.
08 a 09	GET de reservas por usuario y POST de una reserva temporal.
10 a 13	DELETE de los registros temporales y comprobación de 404 después de eliminarlos.

Los POST guardan los nuevos IDs en variables y los DELETE usan esos valores. Ejecuta toda la secuencia para completar la limpieza. La colección está diseñada para los datos de demostración.

Cómo interpretar el quinto método

El PDF exige demostrar cinco métodos HTTP, pero el cambio de estado de reservas desde Angular corresponde al avance 3. Por eso la colección usa PATCH como diagnóstico de un laboratorio temporal; el panel de semana 13 no adelanta la aprobación de reservas.

Verificaciones realizadas

Control	Resultado
Núcleo	30 pruebas aprobadas con node:test y assert.
Contrato HTTP	15 comprobaciones aprobadas; Node cuenta 16 al incluir el test contenedor.
Postman	13 peticiones y 18 verificaciones aprobadas con Newman.
Angular	Compilación de producción completada.
Navegador	CRUD, detalle, filtros, validación, 404, error de red y reintento verificados.
Pantalla móvil	Vista de 390 px revisada; tabla con desplazamiento interno sin desborde de página.

Límite que debes poder explicar

json-server guarda datos pero no ejecuta las reglas del negocio ni autentica usuarios. Las validaciones del núcleo y del formulario no protegen peticiones manuales. Express y JWT se incorporan en la semana 17.

La comprobación se realizó en un entorno Linux con Node 24.22.1. Los scripts están escritos para Windows y Linux; deben ensayarse en la PC real del laboratorio.

GitHub y entrega del avance

El ZIP está preparado para convertirse en el repositorio del grupo. El enlace y la etiqueta no se publican automáticamente: requieren el repositorio real y la integración de los aportes de sus integrantes.

Antes de publicar

Completa el README con los nombres, códigos y cuentas GitHub reales de 3 o 4 integrantes. Confirma que el docente aprobó CampusHub Labs. Cada estudiante debe hacer commits con su propia cuenta y comprender sus cambios.

Crear el repositorio por primera vez

Crea un repositorio vacío en GitHub y copia su URL HTTPS. Reemplaza los textos en mayúsculas de estos comandos por tus datos. Si el grupo ya tiene un repositorio, conserva su historial y usa la guía detallada del ZIP.

```
git init -b main
git config user.name "TU NOMBRE REAL"
git config user.email "TU CORREO DE GITHUB"
git add .
git commit -m "feat: preparar avance 1 CampusHub"
git remote add origin URL_HTTPS_REAL
git push -u origin main
```

Etiquetar la versión que se calificará

Después de integrar los aportes y comprobar que todo funciona:

```
npm run verificar
git tag -a avance-1 -m "Avance 1 - Semana 13"
git push origin main
git push origin avance-1
```

Lista final de entrega

1. Repositorio accesible para el docente: público o con invitación si es privado.
2. README completo y declaración de uso de IA.
3. Etiqueta avance-1 en la versión final.
4. Commits reales de cada integrante.
5. Enlace registrado en el aula virtual antes de la sustentación.
6. Ensayo de arranque en la PC del laboratorio.

La primera instalación depende de Internet y puede exceder los dos minutos de arranque. Prepararla y ensayarla con anticipación. No reutilizar node_modules de Linux en Windows; el lockfile permite instalar las dependencias correctas.

Explicarlo con tu propio código

Presentación breve

CampusHub Labs administra laboratorios del campus. En este avance usamos Angular y TypeScript para el panel, Bootstrap para el diseño y json-server como API simulada. Separamos reglas y estadísticas en un núcleo JavaScript con pruebas. El catálogo permite crear, consultar, editar y eliminar, y el panel también muestra las reservas.

Preguntas que debes practicar

Pregunta	Respuesta clave
¿Por qué reduce empieza con un objeto?	Porque necesitamos tres contadores identificados por estado. Una lista vacía devuelve ceros.
¿Map, Set y map() son iguales?	Map asocia claves y valores; Set evita duplicados; map() transforma cada elemento de un arreglo.
¿Qué recuerda el closure?	El siguiente número del código; padStart lo completa con ceros.
¿Una reserva a las 10:00 choca con otra que termina a las 10:00?	No. Los intervalos son contiguos y se comparan con límites estrictos.
¿Por qué no cambia el catálogo al hacer el ranking?	Se ordena una copia creada con spread.
¿Dónde está Bootstrap?	package.json, angular.json y clases de las plantillas.
¿El mock aplica las validaciones?	No. La validación del servidor corresponde al backend Express posterior.

Cambio en vivo: agregar piso de 1 a 10

Practica localizar la interface, el FormGroup, el input, el validador compartido y los datos de muestra. Añade el campo de forma coherente en todas esas piezas, muestra el valor en el panel y prueba 0, 1, 10 y 11. El recorrido completo está en docs/SUSTENTACION.md.

Declaración de IA

El README declara el uso de ChatGPT/Codex para generar la base, pruebas y documentación, como exige el PDF. Cada integrante debe revisar, ejecutar, modificar y poder explicar el código línea por línea. Esa revisión personal no queda acreditada por generar estos archivos.

Referencias para consultar

Enunciado del profesor: secciones 7, 12, 13 y 14.
Angular: angular.dev/reference/versions y angular.dev/guide/forms/reactive-forms.
Bootstrap: getbootstrap.com/docs/5.3/getting-started/download/.
Las referencias completas y enlaces están en README.md.